Treadmill Hardware Synchronization: Implementation Plan

Window: 16/05 to 30/05/2026 Estimated effort: 8 to 12 hours

Deliverables: `treadmill_sync_firmware.ino`, `treadmill_bridge_node.py`, `calibrate.py`.

1 System overview

The treadmill original equipment lower control board (LCB) retains full responsibility for high-power motor drive. The ESP32 replaces only the console speed-reference signal, injected through optoisolation. Belt velocity is closed-loop controlled against the roller pulse sensor. ROS2 issues start and stop commands and streams a target velocity to the ESP32; the ESP32 streams the measured belt velocity back so the robot locomotion stack can close the outer synchronization loop.

The command path runs from the robot locomotion stack, which publishes a target velocity on `/vel_cmd`, to the bridge node, which forwards it over serial to the ESP32, which drives a PWM speed reference through the optocoupler into the LCB. The feedback path runs from the roller pulse sensor into the ESP32, which computes measured velocity and reports it over serial to the bridge node, which republishes it on `/platform/measured_velocity` for the robot to consume.

The governing design invariant is that the microcontroller never switches motor current. It only sources a low-voltage reference and senses status. All high-power switching is performed by the LCB during normal operation, or by the contactor during an emergency stop.

2 Safety architecture

Three independent stop layers are provided. Each degrades gracefully: a failure at any level is caught by a lower, simpler, more independent level. Layer 1 functions with all software and firmware dead.

Layer	**Mechanism**	**Depends on**	**Triggers on**
1. Physical E-stop	Mushroom NC contact opens the contactor coil, cutting mains to the LCB	Nothing (pure hardware)	Human press
2. Hardware heartbeat	Robot GPIO toggles ESP32 pin 33; loss of toggling ramps the belt to zero	ESP32 firmware only	Robot computer crash or power loss
3. Software watchdogs	Staleness of `/robot/heartbeat` and `/vel_cmd`	ROS2 and serial link	Node, ROS2, or serial failure

Watchdog-triggered stops use the firmware slew limiter (`MAX_ACCEL`), so they decelerate smoothly rather than cutting instantly. The belt never drops out from under the robot except on a deliberate physical E-stop.

3 High-voltage safety and capacitor discharge protocol

Munich mains is 230 V at 50 Hz. After the LCB bridge rectifier the bulk capacitor sits at approximately 325 V DC and remains charged after unplugging. This is lethal. The LCB enclosure and the E-stop box must not be opened under power, and the mains-side work must be performed with a RoboTUM hardware lead or lab supervisor present.

The discharge procedure must be repeated every time before touching the LCB or mains wiring:

1. Unplug from the wall by pulling the plug, not merely switching off power.
2. Wait approximately 60 s. A bleeder resistor may drain the bus, but its presence and health must never be assumed.
3. Identify the large electrolytic bulk capacitor near the rectifier and heatsink.
4. Clip a discharge resistor (5 to 10 kΩ, at least 20 W wirewound, insulated leads) across its positive and negative terminals for approximately 10 s. The time constant is $\tau = RC$; with 10 kΩ and 470 μF, $5\tau \approx 24$ s.
5. Verify with a digital multimeter across the terminals. Do not proceed until the reading is below 30 V, ideally below 5 V. Re-clip if still high.
6. Never short a charged high-voltage capacitor directly with a screwdriver; always discharge through the resistor.

The heatsink, motor leads, and mains-side traces are to be treated as live until metered otherwise.

4 Hardware build sequence

The build proceeds in the order below. The E-stop box is built first; nothing else is powered until it exists and has been verified.

4.1 E-stop interruptor box (mains side, built first)

A small enclosure is inserted into the treadmill mains cable. This requires cutting the cable and wiring in a contactor gated by the mushroom button. The required connections are given below.

Connection	**Detail**
Mains live in	Wall live, through the mains fuse, to the contactor input terminal
Mains live out	Contactor output terminal to the LCB live input
Mains neutral	Wall neutral passes straight through to the LCB neutral
Contactor coil	Energised only while the E-stop normally-closed contact is closed; the coil sits in series with that contact across the mains
E-stop sense	Auxiliary normally-closed contact of the E-stop to ESP32 pin 26 (`ESTOP_SENSE`, active low)

Component and wiring requirements: the contactor has a 230 V AC coil and a current rating at or above the treadmill nameplate (typically 16 A); the mushroom button is latching, 230 V rated, with both a normally-closed and an auxiliary normally-closed contact. All stranded conductors are terminated with ferrules. The mushroom button is mounted on the enclosure exterior within arm's reach of the test area. Before first power-up, verify with a multimeter that pressing the button opens the contactor and drops continuity to the LCB live. This must be confirmed by hand before any software is trusted.

4.2 Signal sniffing to identify the LCB reference type

With the cover on and the unit running, probe only the low-voltage console-to-LCB link, never the power side, using a logic analyzer or oscilloscope while sweeping the console speed. The following must be identified: the speed-reference line, which varies monotonically with speed and is either a PWM signal (fixed frequency, duty cycle encodes speed, MC-2100 class) or an analog signal (0 to reference-voltage DC, MC-60 or MC-70 class); the LCB safety or enable line (magnetic key circuit), which must be satisfied for the board to run; and the LCB signal-ground reference. This step determines whether the RC filter of the next subsection is required.

4.3 Optoisolated signal bridge (PC817)

ESP32 `PWM_OUT_PIN` (pin 25) drives a PC817 LED; the phototransistor reconstructs the reference on the LCB side. The two grounds meet only through the optocoupler and must not be bonded.

Node	Connection
LED drive	ESP32 pin 25, through a 330 Ω series resistor, to PC817 LED anode (pin 1)
LED return	PC817 LED cathode (pin 2) to ESP32 ground
Collector	PC817 collector (pin 4) to the LCB reference rail (for example 5 V)
Emitter	PC817 emitter (pin 3) to the LCB speed-reference input, with a 10 kΩ pull-down to LCB ground
Analog LCB only	Add a 1 kΩ series resistor and a 10 μF capacitor to LCB ground on the emitter node, forming an RC low-pass that smooths the PWM into a DC level (corner near 16 Hz)

The 330 Ω series resistor gives approximately 10 mA from the 3.3 V GPIO. A 20 kHz carrier is used: for analog LCBs the RC filter smooths it to DC; for PWM LCBs the signal is fed through directly and matched to the sniffed mapping.

4.4 Roller speed feedback

The roller reed or optical pulse line is brought to `SPEED_FB_PIN` (pin 27), through a second PC817 if it is referenced to the high-voltage side. The firmware counts pulses per control tick and converts to velocity, scaled by `PULSES_PER_METER`, which is set during calibration.

4.5 Robot heartbeat wire (hardware safety layer 2)

A single wire runs from a robot computer GPIO to ESP32 `HEARTBEAT_PIN` (pin 33). The robot toggles it at approximately 10 Hz. The firmware ramps the belt to zero if edges stop for more than 500 ms while running, independent of ROS2 and the serial link.

4.6 ESP32 pin map

GPIO	Name	Function
25	`PWM_OUT`	PWM speed reference to the PC817 LED
27	`SPEED_FB`	Roller pulse input (input pull-up)
26	`ESTOP_SENSE`	E-stop auxiliary contact, active low
33	`HEARTBEAT`	Robot computer heartbeat, any edge
2	`STATUS_LED`	Onboard status indicator

5 Firmware flash and two-step calibration

The firmware is flashed once, then calibrated in two steps. Each step depends on the previous being correct.

Flashing: open `treadmill_sync_firmware.ino` with Arduino-ESP32 core 3.0 or later, select the board and port, and upload. The placeholder constants run but are inaccurate.

Calibration step 1, physical, sets `PULSES_PER_METER`. Mark the belt, start it with `S` then `V0.3`, run exactly one metre, and stop with `X`. Count the pulses reported over that metre in the telemetry, then set `PULSES_PER_METER` to that count and reflash. Without this being correct the firmware cannot report metres per second accurately, so step 2 would produce a meaningless curve.

Calibration step 2, software, sets `FF_SLOPE` and `FF_INTERCEPT`. Run `python3 calibrate.py -port /dev/ttyUSB0`. The script triggers the firmware duty sweep, fits a straight line of velocity against duty, and prints the feedforward constants. Paste them in and reflash. The gains `KP` and `KI` may then be retuned for clean step tracking.

6 ROS2 integration

The bridge node is run with the following command:

```
python3 treadmill_bridge_node.py -ros-args -p port:=/dev/ttyUSB0
```

Its interface is given below.

Type	**Name**	**Message or service**	**Dir.**
Service	`/platform/start`	`std_srvs/Trigger`	in
Service	`/platform/stop`	`std_srvs/Trigger`	in
Subscriber	`/vel_cmd`	`geometry_msgs/Twist` (uses `linear.x`)	in
Subscriber	`/robot/heartbeat`	`std_msgs/Bool` at approx. 10 Hz	in
Publisher	`/platform/measured_velocity`	`std_msgs/Float32`	out
Publisher	`/platform/tracking_error`	`std_msgs/Float32`	out
Publisher	`/platform/state`	`std_msgs/String`	out
Publisher	`/platform/estop`	`std_msgs/Bool`	out

The robot locomotion stack is required to publish a target belt speed to `/vel_cmd`; subscribe to `/platform/measured_velocity` and adjust gait to the actual belt speed, which closes the outer synchronization loop; publish `/robot/heartbeat` at approximately 10 Hz for software safety layer 3; and optionally drive the ESP32 heartbeat GPIO for hardware safety layer 2. This robot-side work is not yet implemented.

7 Bring-up checklist

1. Build the E-stop box and verify by hand with a multimeter that the contactor drops. No LCB power until this passes.
2. Perform the discharge protocol, then wire the optoisolated signal bridge and roller feedback with the LCB unpowered.
3. Bench-verify on a scope that the ESP32 PWM appears at the LCB speed-reference pin with correct polarity and range.

4. Power up and test that the physical E-stop kills the belt at low speed before trusting any software.
5. Run calibration step 1 and reflash, then calibration step 2 and reflash.
6. Tune `KP` and `KI`. Verify `/vel_cmd` step tracking, watchdog behaviour, and both heartbeat layers.

8 Evaluation criteria and open items

Criterion	**Met by**
Services toggle platform state	`/platform/start` and `/platform/stop` mapped to firmware `S` and `X` state machine
Velocity tracking error minimal under load	Closed-loop PI with feedforward, slew-limited acceleration, and live `/platform/tracking_error`
Physical E-stop drops power instantly with no firmware dependency	Mushroom NC contact opens the contactor coil; the ESP32 only senses the state via the auxiliary contact

Open items before the first live run: confirm the LCB reference type (PWM versus analog) by signal sniffing, which decides whether the RC filter is fitted; measure `PULSES_PER_METER` in calibration step 1; run calibration step 2 and replace the placeholder `FF_SLOPE`, `FF_INTERCEPT`, and gains; and implement the robot-side `/platform/measured_velocity` subscriber and `/robot/heartbeat` publisher, since the bridge and firmware are ready to receive both but nothing on the robot consumes them yet.